Siddhant's AR Work Plan — PastPort India

Role

Siddhant is the **AR Lead** for PastPort India. He owns the phone-based AR experience during the first implementation days and prepares the AR module so it can be connected later to the complete app, which will offer both **AR and VR options**.

The AR module must use the same optimized GLB/GLTF monument model that the normal 3D and optional VR experiences use. Siddhant must not create a separate unrelated AR model or replace the 3D/VR viewer.

Main objective

Build a reliable, beginner-friendly **marker-based AR demonstration**:

Plain Text

```
User opens Camera AR
  -> grants camera permission
  -> points phone at the PastPort marker card
  -> Shaniwar Wada model appears on the marker
  -> user can view/rotate the model and tap simple AR information points
  -> user can return to normal 3D view or later enter VR mode
```

The AR feature is an enhancement. The main application must still work if the camera, AR library, browser, or device does not support AR.

Important file rule

> **Siddhant must not change other people's files without explicit approval from Priyansh and the relevant owner.**

Files Siddhant may create or edit

Use the exact paths below if the project follows the planned React structure:

Plain Text

```
client/src/features/ar/ARViewer.tsx
client/src/features/ar/arConfig.ts
client/src/features/ar/arTypes.ts
```

```
client/src/features/ar/arUtils.ts
client/src/features/ar/README.md
```

If the project uses a `src/` folder instead of `client/src/`, create the same isolated folder under the project's existing frontend source directory, but do not edit unrelated files.

Siddhant may also prepare assets outside the application code and submit them for review:

Plain Text

```
/home/ubuntu/webdev-static-assets/ar/
```

Potential AR assets include the image-target source, marker-card image, model preview, and test screenshots. Large media must not be placed casually in the project's public folder. Priyansh will handle the final approved asset upload/storage process.

Files Siddhant must not modify

Unless Priyansh explicitly approves a coordinated change, Siddhant must not edit:

Plain Text

```
client/src/App.tsx
client/src/main.tsx
client/src/index.css
client/src/pages/*
client/src/components/* outside client/src/features/ar/
client/src/features/vr/*
client/src/features/model-viewer/*
server/*
dizzle/*
shared/*
package.json
pnpm-lock.yaml
.env files
```

He must not change the global Tailwind theme, routes, database, backend, authentication, shared model types, package versions, or VR implementation. If a new dependency is required, he must write the package name and reason in `client/src/features/ar/README.md` and ask Priyansh to install it.

Integration contract with the rest of the team

Siddhant delivers an isolated `ARViewer` component. Priyansh or Shreyas will later connect it to the app route and page layout. Siddhant should provide the following contract:

TSX

```tsx
<ARViewer
  modelUrl="/approved/path/shaniwar-wada.glb"
  targetUrl="/approved/path/shaniwar-wada-target.mind"
  monumentName="Shaniwar Wada"
  onExit={() => void 0}
  onFallback={() => void 0}
/>
```

The exact prop names may be adjusted after the repository is inspected, but the principle must remain: **AR receives the model and target as inputs and does not own application routing, global navigation, or the VR button.**

The final product should expose separate options:

Plain Text

```
View in 3D  -> normal Three.js/R3F viewer
View in AR  -> Siddhant's ARViewer
Enter VR    -> separate WebXR/VR experience using the same model
```

Technology to use

Technology	Siddhant's use
React + TypeScript	Isolated AR component and typed configuration
MindAR.js	Image-target tracking for reliable phone AR
Three.js or the project's existing 3D layer	Render the GLB model on the detected target
GLB/GLTF	Shared monument model format for AR, 3D, and VR
Browser camera API through MindAR	Camera permission and live target detection
HTTPS deployment	Required for camera access on the public demo URL
WebXR	Not Siddhant's first responsibility; VR is a separate optional integration using the same model

Do not start with markerless AR or outdoor GPS-anchored AR. Marker-based AR is the first success criterion because it is more predictable for a one-week beginner project.

Day 1 — Learn the AR flow and create an isolated proof of concept

Learning goals

Siddhant should understand the difference between:

- a normal 3D viewer;
- marker-based AR;
- markerless AR; and
- VR/WebXR.

The immediate goal is not to create a beautiful experience. It is to prove that the phone can detect a printed target and display a simple object.

Work

1. Create a separate branch named `feature/siddhant-ar`.
2. Read the existing project README and inspect the existing 3D model format without editing other modules.
3. Ask AI to explain the project's frontend structure and identify where an isolated AR component can live.
4. Create only the AR feature folder and `ARViewer.tsx`.
5. Test MindAR with a simple cube or placeholder object before using the monument model.
6. Test the flow on Siddhant's Android phone using the local development URL or an HTTPS preview.
7. Record the exact browser, phone model, library version, and result in `client/src/features/ar/README.md`.

Day 1 deliverables

Deliverable	Required result
Branch	`feature/siddhant-ar` exists

Isolated component	`ARViewer.tsx` renders without changing app routes
Camera test	Browser requests camera permission
Target test	Printed/image target is detected or the failure is documented
Proof	Screenshot or short screen recording
Notes	Setup steps and known limitations in AR README

AI prompt for Day 1

Plain Text

```
We are building PastPort India with React and TypeScript.
Create only an isolated component at client/src/features/ar/ARViewer.tsx.
Use MindAR image-target AR if it is already installed; do not modify App.tsx,
main.tsx, global styles, package.json, server files, VR files, or shared files.
Start with a simple placeholder cube.
Include camera-permission, loading, target-not-found, error, and exit states.
Explain every line in beginner-friendly language and give the exact test steps.
```

Day 1 stop rule

If the library or camera setup cannot be made to work within the planned work session, Siddhant must document the error and ask Priyansh for help. He must not install several competing AR libraries or rewrite the project.

Day 2 — Attach the monument model to the target

Work

1. Confirm the approved GLB/GLTF model path with Priyansh.
2. Replace the placeholder cube with the same model used by the normal 3D viewer.
3. Add scale, position, and rotation configuration in `arConfig.ts` rather than hard-coding values throughout the component.
4. Add a loading indicator and a model-failure fallback.

5. Test the target at different distances, angles, lighting conditions, and phone orientations.
6. Keep model size reasonable and report load time and visible performance.
7. Do not modify the shared 3D viewer to make AR work. If the model needs preparation, report the required changes to Priyansh.

Day 2 deliverables

Deliverable	Required result
Shared model use	AR loads the approved GLB/GLTF model
Configuration	Model scale/position/rotation are isolated in `arConfig.ts`
Loading state	User sees progress while the model loads
Failure state	User receives a clear fallback message
Phone test	Model appears on the marker on at least one phone
Evidence	Short video or screenshots at two angles

AI prompt for Day 2

Plain Text

```
Modify only the files inside client/src/features/ar/.
Replace the placeholder cube with this existing GLB model URL: [URL].
Do not edit the shared 3D viewer or any VR file.
Add configurable scale, position, rotation, loading, and error states.
If the model cannot load, show a button callback for normal 3D fallback.
First explain which files you will change, then implement only those changes.
```

Day 3 — Add simple educational AR interactions

Work

1. Add a title such as “Shaniwar Wada — AR View.”
2. Add no more than two or three simple hotspots or labels in AR.

3. Use a small configuration array in `arConfig.ts` for hotspot positions and titles.
4. When a hotspot is selected, call a callback or show a local panel controlled within the AR component. Do not connect directly to the backend or rewrite the global hotspot system.
5. Add a clear “Exit AR” control.
6. Add a “Open 3D View” fallback callback for devices that cannot continue with AR.
7. Keep factual text short. Manmath will provide the verified wording and source IDs.

Day 3 deliverables

Deliverable	Required result
AR title	User knows which monument is displayed
Hotspots	Two or three stable labels/points work
Exit	User can leave AR without a browser dead end
Fallback	User can request the normal 3D experience
Content boundary	No unsourced historical claims are invented in code

Day 4 — Make AR reliable and hand it over

Work

1. Add all expected error states: camera denied, target not found, unsupported browser, model failure, and slow loading.
2. Test portrait and landscape orientation.
3. Test target recognition in bright, dim, near, and angled conditions.
4. Test on at least one Android phone and one desktop browser where a fallback message should appear.
5. Check that AR does not break the normal 3D and future VR paths.
6. Write the integration instructions for Priyansh and Shreyas in `ARViewer.md` or `README.md`.
7. Open a pull request containing only the AR feature folder and approved AR assets.

8. Demonstrate the branch to Priyansh and Shreyas before merging.

Day 4 deliverables

- `ARViewer` component;
- `arConfig` and typed AR data;
- AR README with setup/test instructions;
- target asset and model requirements;
- screenshots or recording;
- known limitations;
- fallback behaviour;
- pull request with a clear description.

AI prompt for Day 4

Plain Text

```
Review only the files in client/src/features/ar/.
Do not edit any files outside that folder.
Add robust handling for camera permission denial, target not found,
unsupported browser, model loading failure, slow loading, exit, and fallback.
Do not add new features. Explain the smallest changes and provide a manual
Android test checklist for a beginner.
```

Day 5 onward — Support, do not take over, the shared application

After the AR pull request is merged, Siddhant’ s role changes from primary builder to AR maintainer and tester.

Day	Siddhant’ s support task
Day 5	Help Priyansh/Shreyas connect the AR component to the monument page without taking over their route or layout files
Day 6	Test AR on the public HTTPS deployment, optimize model loading, and prepare recorded AR backup

Day 7	Verify the final demo, answer AR technical questions, and confirm that AR and VR buttons are separate and understandable

Siddhant may recommend changes to other files, but the owner of those files must apply them. If a change is necessary in a shared file, create a GitHub issue describing the exact line, reason, risk, and test result instead of editing it directly.

GitHub workflow for Siddhant

Plain Text

```
Pull latest main
  -> create/update feature/siddhant-ar
  -> make only AR-folder changes
  -> run app and phone test
  -> commit one logical change
  -> open pull request
  -> request review from Priyansh and Shreyas
  -> fix review comments only in AR scope
  -> merge after approval
```

Suggested commits:

Plain Text

```
feat(ar): add isolated MindAR viewer shell
feat(ar): load approved monument GLB on image target
feat(ar): add AR loading and camera error states
feat(ar): add configurable educational hotspots
 test(ar): document Android marker recognition results
```

Definition of done

Siddhant’ s AR work is complete when:

1. The AR component is isolated in the approved feature folder.
2. It uses the same monument model as normal 3D and planned VR.
3. It detects a printed target on at least one tested phone.
4. The model appears at a sensible scale and does not immediately disappear.
5. Two or three educational hotspots work.

6. Camera denial, target failure, model failure, and unsupported-device states are handled.
7. The user can exit AR and return to normal 3D.
8. No unrelated files were changed.
9. The pull request is reviewed and merged.
10. A backup screen recording exists for the final presentation.

What Siddhant must not promise

Siddhant should not promise perfect markerless AR, outdoor GPS-level placement, photorealistic historical accuracy, universal phone compatibility, or a full VR implementation. The final product has both AR and VR options, but Siddhant owns the **AR module**; the VR mode remains a separate experience using the same 3D asset and a normal 3D fallback.